Received: 28 September 2019 | Revised: 27 November 2019 | Accepted: 25 December 2019
DOI: 10.1002/cpe.5674

SPECIAL ISSUE PAPER

WILEY

Dynamic resource provisioning for workflow scheduling under uncertainty in edge computing environment

Xiaolong Xu[1,2,3,4] | Hao Cao[1] | Qingfan Geng[1] | Xihua Liu[1] | Fei Dai[5] | Chuanjian Wang[6]

[1]School of Computer and Software, Nanjing University of Information Science and Technology, Nanjing, China

[2]Jiangsu Engineering Center of Network Monitoring, Nanjing University of Information Science and Technology, Nanjing, China

[3]Jiangsu Collaborative Innovation Center of Atmospheric Environment and Equipment Technology (CICAEET), Nanjing University of Information Science and Technology, Nanjing, China

[4]Engineering Research Center of Digital Forensics, Ministry of Education, Nanjing, China

[5]School of Big Data and Intelligence Engineering, Southwest Forestry University Kunming, China

[6]College of Information Science and Technology, Shihezi University Xinjiang, China

Correspondence
Fei Dai, School of Big Data and Intelligence Engineering, Southwest Forestry University, Kunming, China.
Email: flydai.cn@gmail.com

Funding information
National Natural Science Foundation of China, 61702277, 61702442; Priority Academic Program Development of Jiangsu Higher Education Institutions (PAPD)

Summary

Edge computing, an extension of cloud computing, is introduced to provide sufficient computing and storage resources for mobile devices. Moreover, a series of computing tasks in a mobile device are set as structured computing processes and flows to achieve effective management by the workflow. However, the execution uncertainty caused by performance degradation, service failure, and new service additions remains a huge challenge to the user's service experience. In order to address the uncertainty, a software-defined network (SDN)-based edge computing framework and a dynamic resource provisioning (UARP) method are proposed in this paper. The UARP method is implemented in the proposed framework and addresses the uncertainty through the advantages of SDN. In addition, the nondominated sorting genetic algorithm-III is employed to optimize two goals, that is, the energy consumption and the completion time, to obtain balanced scheduling strategies. The comparative experiments are performed and the results show that the UARP method is superior to other methods in addressing the uncertainty, while reducing energy consumption and shortening the completion time.

KEYWORDS

edge computing, SDN, uncertainty, workflow scheduling

1 | INTRODUCTION

With the development of Internet of Things, a large number of applications and connections need to be supported by a network with high scalability, reliable data transmission, low latency, and high bandwidth. Traditional cloud computing emerges as a viable paradigm and migrates computing tasks from mobile devices to cloud data centers with abundant computing and storage resources.[1-4] Furthermore, in order to achieve effective utilization of resources, the workflow is introduced to simulate and manage the computing tasks in the applications automatically in cloud computing. However, cloud computing is incapable of neglecting the data transmission delay while solving the problem of insufficient local resources in mobile devices. To meet the requirements of the network, the European Telecommunication Standards Association established the Mobile Edge Computing (MEC) Specification Working Group in 2014 and formally announced the promotion of MEC standardization. The basic idea of MEC is to migrate the cloud computing platform from the core network to the edge of the mobile access network, so as to realize the flexible utilization of computing and storage resources. Moreover, MEC is able to reduce the end-to-end delay of information transmission and makes full use of the computing power of the network, thus improving the quality of service(QoS) and user experience.[5,6]

 © 2020 John Wiley & Sons, Ltd.

In the edge network, the traditional underlying network architecture is incapable of meeting various new needs, and the complex configuration of equipment is difficult to iterate.[7] In view of the challenges, software-defined network (SDN), which has the ability to make the network more flexible and dynamic through a control plane with logic centralization, is introduced.[8] In SDN, the control plane is separated from the data plane and the direct programming control of the network is realized through a unified and open interface.[9] Fortunately, SDN in centralized network management brings some inspirations to latency-sensitive networks. Although there are many different routes between the mobile devices and the servers in SDN design, the performance of the whole network depends on the ability of the controller to determine routes.[10] With the increase of computing requests, these computing requests need to be reasonably allocated to different servers to reduce latency, thereby ensuring overall network performance.[11]

In our paper, SDN is integrated with MEC to determine the workflow scheduling, and complex service chain scenes are able to be realized through traffic-oriented rules. SDN is applied to manage distributed MEC networks. In practical applications, the SDN controllers are considered to be deployed near the edge nodes,[12] and the edge controllers migrate the information calculation to the network devices, so as to decentralize the computing tasks and processing pressure of the centralized controller. In this case, one SDN controller is designed to monitor several edge nodes around it. Moreover, the SDN controllers communicate with different edge controllers to determine which MEC servers are used to serve the requests of client applications in a certain service type. Therefore, SDN is used in MEC to provide a unified control plane interface and retrieve network-related information,[13] which is used for intelligent workflow scheduling across networks.

Despite the introduction of SDN, the resource allocation in edge computing is still complex. There are some unpredictable situations in workflow scheduling process, such as resource conflicts, service failures, performance degradation, and other unstable factors, defined by the uncertainty, which makes the previous static scheduling strategies no longer applicable and requires a kind of new real-time strategies.[14] Experiments carried out by Dejun Jiang et al show that the performance of the same instance varies in different repetitions and experimental execution, and the variation results are difficult to predict.[15,16] Considering the control information between edge nodes, the uncertainty in the edge computing environment is more complex than traditional cloud computing.[17] To detect the state of peripheral edge nodes in workflow scheduling, SDN controllers need to collect information regularly from edge nodes under their control and establish initial workflow scheduling strategies. In addition, when the uncertainty arises, the SDN controller needs to transfer information to other controllers to rearrange the time and the transmission cost of control information is not negligible. With the increase of the distance between the controllers and the edge nodes, the transmission cost also increases.

In view of the above challenges, new optimal workflow scheduling strategy needs to be determined while ensuring the continuity of equipment services. Meanwhile, the SDN controllers synchronize with each other and notifies all edge nodes of the abnormal states to conduct new strategies. Therefore, a dynamic resource provisioning method for workflow scheduling in the edge computing environment, called UARP, is proposed in this paper. The purpose of this method is to optimize the completion time and energy consumption in the edge computing environment while addressing the uncertainty in SDN-based edge computing framework.

The main contributions of this paper are summarized as follows:

(1) A SDN-based edge computing framework and a dynamic resource provisioning method are proposed to address the uncertainty in the workflow scheduling.
(2) A multiobjective optimization problem is defined in the workflow scheduling, and nondominated sorting genetic algorithm-III (NSGA-III)[18] is adopted to search for the balanced scheduling strategies.
(3) The simulation experiments are used to verify the effectiveness of our method. The results show that this method effectively solves the uncertainty problem and reduces energy consumption and execution time at the same time.

The remainder of this paper is organized as follows. Section 2 gives a framework and formal descriptions. Section 3 describes the method we use. In Section 4, experiments show the performance of our method. Section 5 describes the relevant work. The conclusion is summarized in Section 6.

2 | SYSTEM MODEL AND PROBLEM FORMULATION

In this section, the SDN-based edge computing framework and formalized concepts are described. Key parameters and descriptions are given in Table 1.

2.1 | A SDN-based edge computing framework

In our paper, SDN is introduced to reduce the complexities of control functions in edge nodes, and a SDN-based edge computing framework is proposed.[19,20] This proposed framework is divided into four layers and is mainly used to address the uncertainty of workflow scheduling. Control

TABLE 1 Key terms and descriptions for problem formulation

Notation	Description
V	The computing tasks collection in a workflow
M	The number of computing tasks
N	The number of edge nodes
$v_{m,i}$	The ith task in mth workflow
p	The set of the predecessor tasks
s	The set of the successor tasks
x_m	The set of strategies of mth workflow
$\mathrm{WT}(V_m)$	The completion time of m-th workflow
$\mathrm{CM}(V_m)$	The total energy consumption of mth workflow
$\mathrm{SUC}(V_m)$	The success rate of mth workflow
$\mathrm{DT}(V_m)$	The deadline of mth workflow

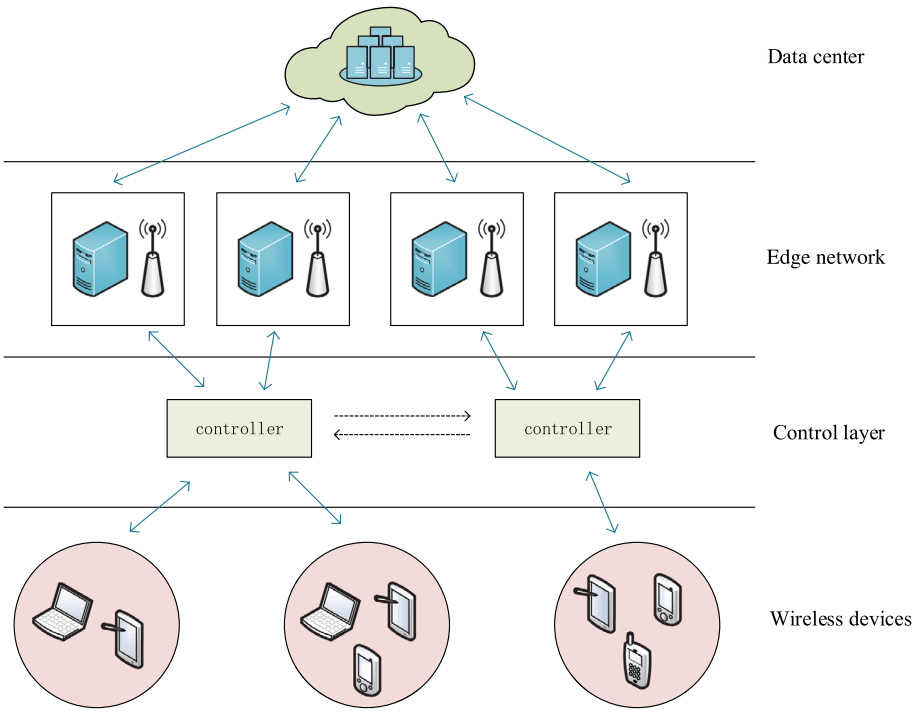


FIGURE 1 An software-defined network-based edge computing framework

functions are usually separated from the data plane and considered as a single layer when SDN is introduced. Figure 1 shows the specific framework. In this framework, the mobile devices are divided into a single layer. When mobile devices have insufficient computing resources to execute the tasks in the workflow, mobile devices send service requests to the edge nodes to get enough resources. The separated control layer has multiple controllers and these controllers implement control functions of one or more edge nodes. Besides, these controllers need to exchange messages periodically for synchronization. This strategy ensures that edge nodes are logically centralized, enabling effective management of resources on edge nodes. The control layer determines the strategies of workflow scheduling according to different optimization goals. Then, the tasks are transferred to the specified edge nodes and data center for processing.

In addition to separating the data layer from the control layer, SDN also supports software programming and this is due to OpenFlow. OpenFlow is a protocol that describes the standard of information exchange and interfaces between the controllers and switches. Edge nodes in different locations can be connected through switches which support OpenFlow and exchange messages.[21] The model in which the OpenFlow switches are connected to SDN controllers is shown in Figure 2. Moreover, the model is also a detailed description of the middle two layers in the proposed SDN-based edge computing framework. What is more, due to the programmability of SDN, it enables mobile devices to be plug-and-play.

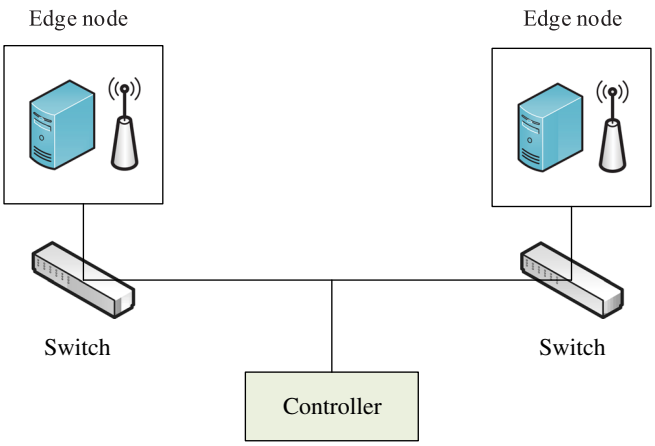


FIGURE 2 Connect edge nodes using software-defined network

2.2 | Problem formulation

In this subsection, the completion time and energy consumption of the workflow are two main optimization goals. Meanwhile, the success rate is introduced to evaluate the performance of UARP method in terms of processing uncertainty.

2.2.1 | Basic concepts

In this paper, there is a sequential relationship between the pending tasks, and the workflow, denoted as $G_m(V_m, T_m)$, is introduced to model this association. $V_m = \{v_{m,1}, v_{m,2}, \dots, v_{m,M}\}$ represents the set of computing tasks on the mth mobile device, and $T_m = (v_{m,i}, v_{m,j})$ represents the sequential relationship between these computing tasks. The task $v_{m,j}$ is only executed when all predecessor tasks are completed. The specific relationships between computing tasks are defined as follows:

$$p(v_{m,i}) = \{v_{m,j} | (v_{m,j}, v_{m,i}) \in T_m\}, \tag{1}$$

$$s(v_{m,i}) = \{v_{m,j} | (v_{m,i}, v_{m,j}) \in T_m\}, \tag{2}$$

where $p(v_{m,i})$ represents the set of all predecessor tasks, and $s(v_{m,i})$ represents the set of all successor tasks.

At some point, suppose there are N edge nodes that provide services for mobile devices, denoted as $Z = \{z_1, z_2, \dots, z_N\}$. A two-tuple $v_{m,i} = (x_{m,i,k}, d_{m,i})$ is introduced, where $x_{m,i,k}$ represents the scheduling strategy of the task $v_{m,i}$, and $d_{m,i}$ represents the distance between $v_{m,i}$ and the edge node serving $v_{m,i}$. The specific scheduling strategy $x_{m,i,k}$ is defined by:

$$x_{m,i,k} = \begin{cases} 1, & \text{if } v_{m,i} \text{ is transmitted to } z_k, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Depending on the scheduling strategy, $v_{m,i}$ is transmitted to z_k, otherwise it is served by the data center.

2.2.2 | Completion time analysis

The completion time of the tasks includes the execution time, the transmission time and the synchronization time between the controllers. First, tasks in the workflow need to be transmitted to the edge nodes. According to the scheduling strategy $x_{m,i,k}$, the distance $d_{m,i}$ between the edge node z_k and mobile device is obtained. Considering the transmission time is affected by the distance and the size, the transmission time of the task $v_{m,i}$ is calculated by:

$$\mathrm{OT}(v_{m,i}) = b_{m,i} \cdot \frac{d_{m,i}}{\mathrm{BA}}, \tag{4}$$

where BA represents the rate at which the unit size task transmits unit distance and $b_{m,i}$ represents the size of $v_{m,i}$.

The execution time of tasks are affected by the size. When one edge node is able to provide sufficient resources for a task, the execution time is defined as:

$$\mathrm{ET}(v_{m,i}) = \frac{b_{m,i}}{B_k}, \tag{5}$$

where B_k represents the processing capability of the edge node z_k, which provides services for $v_{m,i}$.

Then, for the first task of each workflow, the start time is determined and expressed as:

$$\mathrm{ST}(v_{m,1}) = 0. \tag{6}$$

Since the flow chart of the workflow is directly acyclic, a task is allowed to start execution after all predecessor tasks have been completed.[22] Therefore, the start time of the task $v_{m,i}$ is calculated by:

$$\mathrm{ST}(v_{m,i}) = \mathrm{ST}(p(v_{m,i})) + \mathrm{OT}(p(v_{m,i})) + \mathrm{ET}(p(v_{m,i})). \tag{7}$$

Meanwhile, the completion time $\mathrm{WT}(v_{m,i})$ of $v_{m,i}$ is also be known.

$$\mathrm{WT}(v_{m,i}) = \mathrm{ST}(p(v_{m,i})) + \mathrm{ET}(p(v_{m,i})). \tag{8}$$

Suppose there are M tasks in the workflow, and then the M-th task $v_{m,M}$ is the last task of this workflow. When $v_{m,M}$ is completed, the completion time of the workflow $\mathrm{WT}(V_m)$ is calculated.

2.2.3 | Energy consumption analysis

Energy consumption is another important optimization goal in this paper. When a task is transmitted to an edge node, both the execution energy consumption and transmission energy consumption need to be considered. Besides, the energy consumption of the controller synchronization is not negligible and determined by the number of controllers.

According to the scheduling strategies, the transmission energy consumption of the workflow V_m is calculated by:

$$\mathrm{CMT}(V_m) = \sum_{i=1}^{M} \sum_{k=1}^{N} x_{m,i,k} \cdot ct(v_{m,i}, z_k), \tag{9}$$

where $ct(v_{m,i}, z_k)$ represents the energy consumption between $v_{m,i}$ and z_k, and increases as the distance increases.

Similarly, the execution energy consumption is determined by the size of the tasks and calculated by:

$$\mathrm{CME}(V_m) = \sum_{i=1}^{M} \sum_{k=1}^{N} x_{m,i,k} \cdot b_{m,i} \cdot ce(z_k), \tag{10}$$

where $ce(z_k)$ represents the execution energy consumption per unit size of data on z_k.

Then, assume that there are U controllers in proposed SDN-based edge computing framework, the controllers' synchronous energy consumption is calculated.

$$\mathrm{CMS}(V_m) = \sum_{i=1}^{U} \sum_{j=1}^{U} h_{i,j} \cdot cs_{i,j}, \tag{11}$$

where $h_{i,j}$ represents whether ith controller and jth controller exchange synchronization messages, and $cs_{i,j}$ represents the synchronization energy consumption of these two controllers.

In summary, the total energy consumption of the workflow V_m is denoted as:

$$\mathrm{CM}(V_m) = \mathrm{CMT}(V_m) + \mathrm{CME}(V_m) + \mathrm{CMS}(V_m). \tag{12}$$

2.2.4 | Success rate analysis

The above two optimization goals are calculated under ideal conditions. However, it is difficult to achieve the ideal situation in actual scenarios. For each workflow, a deadline DT is given. As long as the completion time of the workflow is less than DT, the QoS is considered to meet the requirements. DT is denoted as:

$$\mathrm{DT}(V_m) = \alpha \cdot \mathrm{WT}(v_{m,M}). \tag{13}$$

This formula represents that DT is α times the ideal completion time of the workflow. Therefore, it is obvious that $\alpha > 1$, and α is adjusted according to the requirements of the QoS.

Suppose the task $v_{m,i}$ is on a critical path of the workflow. Based on the critical path algorithm, the deadline of $v_{m,i}$ is defined by:

$$\mathrm{DT}(v_{m,i}) = (\mathrm{WT}(v_{m,i}) - \mathrm{ST}(v_{m,p})) \cdot \frac{\mathrm{WT}(v_{m,q}) - \mathrm{ST}(v_{m,p})}{\mathrm{WT}(v_{m,i}) - \mathrm{ST}(v_{m,p})}, \tag{14}$$

where $v_{m,p}$ represents the first task on this critical path, and $v_{m,q}$ represents the last task on this critical path.[22]

Once the uncertainties are generated, the affected tasks, denoted as $\mathrm{AF}(V_m) = (af_{m,1}, af_{m,2}, \ldots, af_{m,T})$, are rescheduled and new strategies are generated. In the new strategies, the actual completion time of $af_{m,i}$ is $rt(af_{m,i})$. If $rt(af_{m,i}) < \mathrm{DT}(af_{m,i})$, $af_{m,i}$ is considered to meet the QoS requirements. Then, we define a variable $k(af_{m,i})$ to describe the relationship.

$$k(af_{m,i}) = \begin{cases} 1, & rt(af_{m,i}) < \mathrm{DT}(v_{m,i}), \\ 0, & \text{otherwise}. \end{cases} \tag{15}$$

Therefore, the success rate in addressing the uncertainty is formulated by:

$$\mathrm{SUC}(V_m) = \frac{1}{T} \cdot \sum_{i=1}^{T} k(af_{m,i}). \tag{16}$$

2.2.5 | Object function

In this paper, the goal of workflow scheduling is to find a optimal strategy and the success rate is used to assess the performance of addressing the uncertainty. The multiobjective optimization problem is defined by:

$$\min \mathrm{WT}(v_{m,M}), \mathrm{CM}(V_m). \tag{17}$$

$$s.t.\ \mathrm{WT}(V_m) \leq \mathrm{DT}(V_m), \tag{18}$$

where Equation (17) represents that the completion time and the energy consumption are minimized in the workflow scheduling. The constraint represents that the completion time of the mth workflow should be less than the given deadline.

3 | A DYNAMIC RESOURCE PROVISIONING METHOD

3.1 | Workflow scheduling optimization using NSGA-III

In this section, the proposed dynamic resource provisioning method optimizes multiple optimization goals and finds the optimal workflow scheduling strategy. In order to address this multiobjective problem, the NSGA-III is adopted. For multiple optimization goals, many multiobjective optimization algorithms are not ideal due to the increase of dimension. However, NSGA-III is able to perform global search faster and more accurately while it has strong robustness. More importantly, NSGA-III replaces the crowding distance with a reference point on the basis of NSGA-II to ensure the diversity of the population and avoid the local optimal solutions.

3.1.1 | Encoding and fitness function

The scheduling strategies of the workflow need to be encoded before using NSGA-III. In the genetic algorithm (GA), the scheduling strategy for each task in the workflow is viewed as a gene, and the gene uses integer to indicate which edge node the task is migrated. A chromosome, which is also called an individual, consists of genes corresponding to all tasks in the workflow. To ensure the diversity of the next generation, GA adopts the fitness function. Two fitness functions of the completion time and energy consumption are given in Equations (8) and (12).

3.1.2 | Initialization

Besides, the size of the population S_p and the number of iterations N_{it} need to be determined in advance. The crossover probability P_c and the mutation probability P_m are also key parameters in GA, and these two parameters distinguish GA from other algorithms.

3.1.3 | Crossover and mutation

In NSGA-III, the crossover is to accelerate the convergence of GA, and new chromosomes are generated by exchanging partial gene fragments of two chromosomes. Figure 3 shows the process of chromosome crossover. The chromosome chr_a and chr_b exchange the genes i_3 and i_4, respectively. The mutation is another important operation in GA. By modifying some genes in a chromosome, GA quickly obtains a chromosome with higher fitness value. Moreover, the mutation also prevents early convergence. Figure 4 shows the process of chromosome mutation. The gene i_3 in the chromesome chr_a is replaced by a new gene with different value.

After the initial population Q_p is operated by the genetic operators, including the crossover probability and the mutation probability, new population T_p of size S_p is generated. Then, Q_p and T_p are combined to generate parent population QT_p of the size $2S_p$.

3.1.4 | Selection

For further operation, parent population QT_p is nondominated and the individuals are divided into different nondominated levels, denoted as $LE_i = 1, 2, \ldots$. According to the hierarchy from small to large, the individuals are selected and appended to the offspring population QT_{ch} until the size of QT_{ch} is equal to S_p. However, since the number of individuals at the current highest level causes QT_{ch} to be greater than S_p, only a part of the individuals at this level are selected.

To correlate the reference points, the fitness functions need to be first scalarized. Afterward, the minimum values of the fitness functions, including the completion time WT_m and the energy consumption CM_m, are calculated. The minimum values of the fitness functions are also called ideal points in NSGA-III. The scalar formulas are defined as:

$$WT^t = WT - WT_m. \tag{19}$$

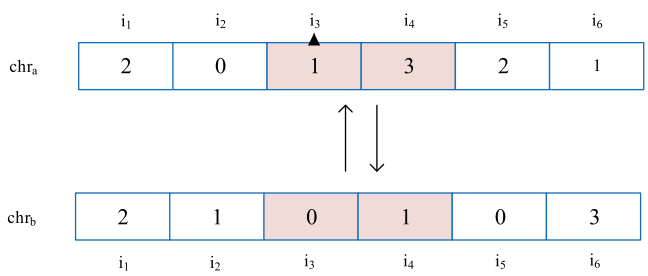


FIGURE 3 Process of chromesome crossover

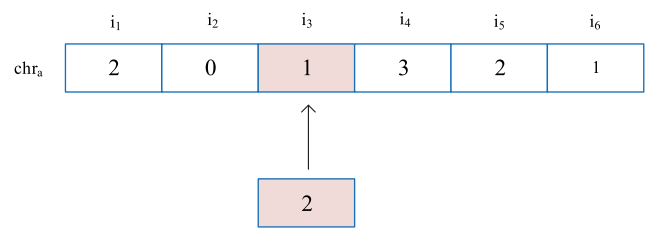


FIGURE 4 Process of chromesome mutation

$$\mathrm{CM}^t = \mathrm{CM} - \mathrm{CM}_m. \tag{20}$$

Besides, the extreme points also need to be calculated, and the extreme point of each target is mapped to the coordinate axes. The corresponding coordinate values on coordinate axis are the intercepts and are recorded as σ_{wt}, σ_{cm}. After the extreme points and minimum values are determined, the fitness functions are normalized:

$$\mathrm{WT}^n = \frac{\mathrm{WT}^t}{\sigma_{\mathrm{wt}}}. \tag{21}$$

$$\mathrm{CM}^n = \frac{\mathrm{CM}^t}{\sigma_{\mathrm{cm}}}. \tag{22}$$

The values of the normalized functions are limited to no more than 1, and the normalization is prepared to associate the individuals to the reference points. The reference points in NSGA-III are generated according to certain rules. Each individual is connected to the reference point that is closest to it, and the distance refers to the vertical distance from the individual to the line between the reference point and the origin.

Finally, the individuals in the offspring population are filtered and some reference points are deleted. A reference point is associated with one or more individuals, or it is not associated with any individual. After nondominated sorting, the size of unfiltered offspring population exceeds S_p. For convenience of description, the current highest nondominated level is denoted as fth level, and the number of individuals connected to this reference point is denoted as po. po does not contain individuals at the fth level. If $po = 0$, and there is an individual at fth level connected to the reference point, the individual closest to the reference point is appended to the offspring population, otherwise the reference point is deleted. If $po > 0$, the individual closest to the reference point is also appended to the offspring population.

3.2 | Finding the Optimal Strategy Using SAW and MCDM

To select the optimal workflow scheduling strategy, simple additive weighting (SAW) and multiple criteria decision-making (MCDM) are adopted to assess the overall performance of the strategies. The two optimization goals, including the energy consumption and the completion time are given different weights. The utility function is defined as:

$$W = \lambda_{wt} \cdot \mathrm{WT}^n + \lambda_{cm} \cdot \mathrm{CM}^n (\lambda_{wt} = \lambda_{cm} = 0.5). \tag{23}$$

where λ_{wt}, λ_{cm} represent the weight of the completion time WT, the energy consumption CM, respectively. Through the setting of weights, the preferences of different goals are determined.

The selection of population is shown in Algorithm 1. First, the initial population Q_p, the size of population S_p and the number of iterations N_{it} are the inputs to the algorithm. After the crossover and mutation operations, new parent population QT_p is generated. Afterward, QT_p performs a series of selection operations and generates multiple balance strategies. Finally, the strategy X_n is output and SDN implements the scheduling with X_n.

3.3 | Addressing the uncertainty in workflow scheduling

In edge network, mobile devices are dynamic, and the resources of edge nodes, including computing resources and storage resources, are also dynamically allocated. The scheduling strategies of the workflow are first generated according to the set optimization goals. However, due to the addition of new mobile devices and performance degradation of servers, the initial strategies no longer meet the need of users. In response to these uncertainties, the proposed SDN-based edge computing framework is capable of discovering them and generating new scheduling strategies.

Performance degradation is a major cause of generating uncertainty. Due to network congestion and resource competition, the tasks to be executed and their subsequent tasks are determined to be incapable of performing according to the old scheduling strategies. SDN finds the above situation and then send OpenFlow messages to notify other edge nodes. The affected tasks are created as a new workflow and new scheduling strategies are regenerated. Besides, the controllers periodically exchange status information about the edge nodes they manage, which help to identify and address uncertainties in advance. Although SDN is able to directly integrate new services and mobile devices without manual configuration, the addition of new mobile devices also leads to more intense competition for computing and storage resources on edge nodes. When the tasks are not completed as expected, the users get a poor service experience. Therefore, regenerating new scheduling strategies is a viable solution. After the introduction of SDN, changes in edge nodes are easily learned by other edge nodes, which allows SDN to quickly detect and generate new scheduling strategies to address the uncertainty.

Algorithm 1. Selection of the population

Input: Q_p, S_p, N_{it}
Output: X_n

```
Calculate WT(V_m), CM(V_m) by Equations (9) and (13)
Initial the population Q_p
Obtain QT_p through crossover and mutation operation
for j = 1 to N_it do
    Nondominated sorting QT_p to find LE_1, LE_2, ...
    Initial QT_ch and Set i = 1
    while QT_ch ≤ S_p do
        QT_ch = QT_ch ∪ LE_i
        i = i + 1
    end while
    QT_ch = QT_ch − LE_{i−1}
    Normalize QT_ch and calculate reference points
    Associate each member of population M^t to reference point
    while QT_ch < S_p do
        Choose an individual from fth level
    end while
    j = j + 1
end for
Pick out the optimal scheduling strategy X_n Using SAW and MCDM
return X_n
```

3.4 | Method overview

In this paper, a dynamic resource provisioning method is proposed to address the uncertainty while optimizing the completion time and the energy consumption. This scheduling problem is a multiobjective optimization problem and NSGA-III is adopted to solve this problem. First, multiple strategies are encoded and the fitness functions are given. Then, the nondominated sorting and reference points are implemented to select the scheduling strategies. Finally, the optimal scheduling strategy of the workflow is selected by the utility function.

The process of the proposed method is expounded in Algorithm 2. First, when the uncertainty is generated, SDN captures the abnormal states of the edge nodes and notifies all edge nodes. Then, the deadline is calculated and the scheduling strategy X_{new} for new workflow is regenerated using Algorithm 2. Finally, SDN implements X_{new} to address the uncertainty.

Algorithm 2. Dynamic resource provisioning

Input: Q_p, S_p, N_{it}, X_n
Output: X_{new}

```
SDN-based framework captures the abnormal states and notifies each edge node
Calculate DT(V_m) by X_n
Generate new workflow with being executed and unexecuted tasks
Reschedule the new workflow and select the population
Calculate SUC(V_m) by Equation (16)
Pick out new scheduling strategy X_new
return X_new
```

4 | EXPERIMENTAL RESULTS AND ANALYSIS

In this section, some experiments are conducted to verify the performance of our proposed method UARP. First, simulated experimental environment is established and some simulated parameters are described. Then, some comparative methods are introduced to compare with our method, and the experimental results are analyzed.

4.1 | Experiment setup

In our experiment, three methods are chosen to compare with UARP to confirm the superiority of UARP. The three methods are Benchmark, FF (First Fit algorithm) and WF (Worst Fit algorithm). The detailed descriptions of these two methods are given below.

(1) Benchmark: The tasks in the workflow are migrated to edge nodes based on the scheduling strategies. However, once the uncertainty is generated, new scheduling strategies are not regenerated. The tasks are executed in accordance with the previous strategies until all tasks are completed, so they may exceed the given deadline.
(2) FF: The tasks in the workflow in turn require the edge nodes that meet the requirements to provide services until all tasks are migrated. Moreover, new scheduling strategies are generated when the uncertainty arises.
(3) WF: The tasks in the workflow are migrated to the edge node with the most resources to reduce resource competition. However, this method may take more time and consume more energy.

The workflow of different scales are used in our experiments, and the number of tasks for the workflow is 10, 15, 20, 25, 30, 35, respectively. When performing the specified tasks, the processing power of the edge nodes are reduced to simulate performance degradation of edge nodes. Moreover, new edge node entities are created in the edge network and new services are provided at a specified point in time. To simulate the edge nodes with different processing capabilities, different numbers of VMs are set in these edge nodes. All methods in this paper are implemented on a desktop PC with Intel Core i7-7700HQ 2.80 GHz processors and 8 GB RAM. Some other simulation parameters are illustrated in Table 2.

4.2 | Performance evaluation of UARP

The multiobjective optimization problem proposed in UARP optimizes overall energy consumption and completion time. In order to find the optimal scheduling strategy, SAW and MCDM, illustrated in Section 3, search for multiple solutions generated by UARP. Figure 5 shows the utility values for workflows of different task scales, and the number of solutions generated each time is different. Based on the comparison of utility values, the most balanced scheduling strategy is the solution with the highest utility value. For example, solution 2 is the optimal strategy in Figure 5A.

4.3 | Comparison analysis

In this subsection, the performance of UARP, Benchmark, FF and WF are compared and analyzed. The completion time and the energy consumption are two important optimization goals. The success rate is the main metric for judging the performance of addressing uncertainty.

TABLE 2 Parameter settings

Parameter description	Value
The bandwidth between mobile devices and edge nodes BA	1000 mbps
The processing power of VM pro	2000 MHZ
The number of the edge nodes	20
The power rate of the start of VM θ_s	0.2 KW
The power rate of the idle VM θ_{id}	0.03 KW
The power rate of operating VM θ_{op}	0.05 KW

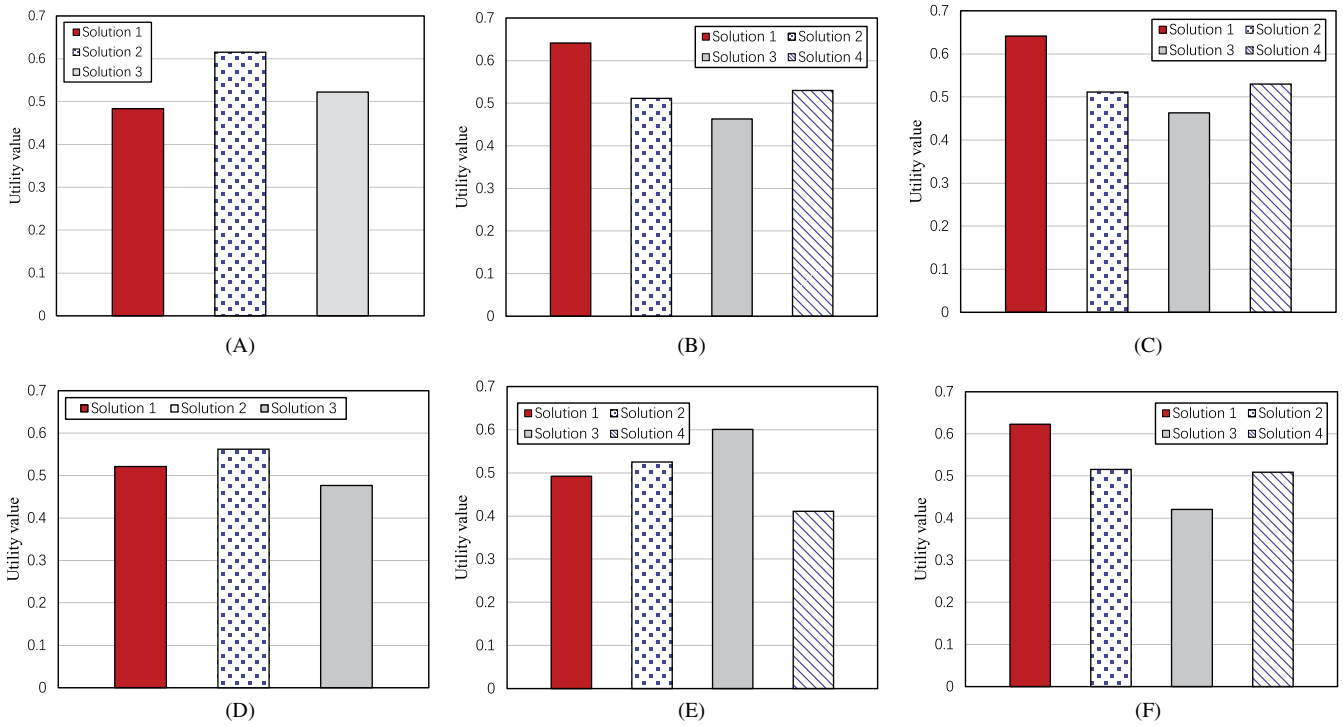
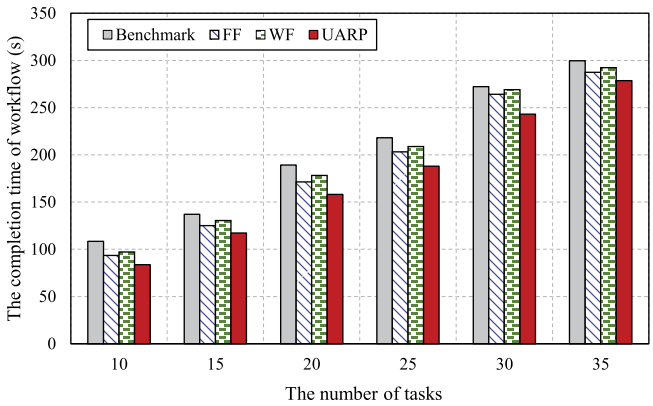


FIGURE 5 The utility value generated by UARP of the workflow with different task scales. A, number of mobile devices = 10; B, number of mobile devices = 15; C, number of mobile devices = 20; D, number of mobile devices = 25; E, number of mobile devices = 30; F, number of mobile devices = 35

FIGURE 6 Comparison of the completion time.



4.3.1 | Comparison on completion time

Figure 6 shows the completion time of the workflows with different number of tasks. In our experiment, Benchmark consumes more time than FF and WF. It means that new scheduling strategies are superior to the initial strategies. Obviously, compared to the other two methods, UARP also has a shorter completion time, which is due to faster and better scheduling strategies. When the uncertainty arises, SDN quickly captures this abnormal state and notifies other edge nodes. New scheduling strategies are also generated, and all edge nodes are aware of the new strategies. The time tap between these three methods is huge, and our method allows users to get a better service experience.

4.3.2 | Comparison on energy consumption

Figure 7 shows the comparison of total energy consumption for workflows with different number of tasks. In our framework, the synchronization of the SDN controllers allows all edge nodes in the edge computing system to acquire the states of other edge nodes, and the globally optimal

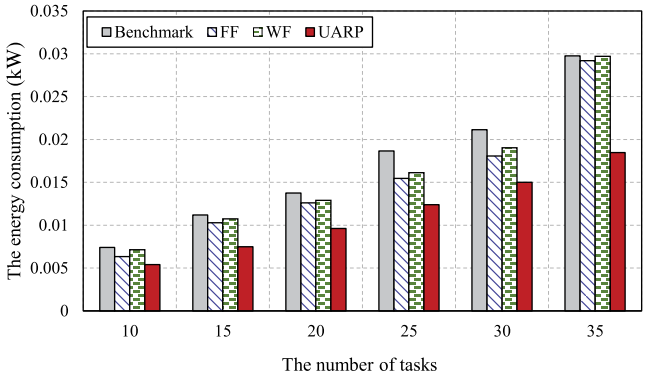


FIGURE 7 Comparison of the energy consumption.

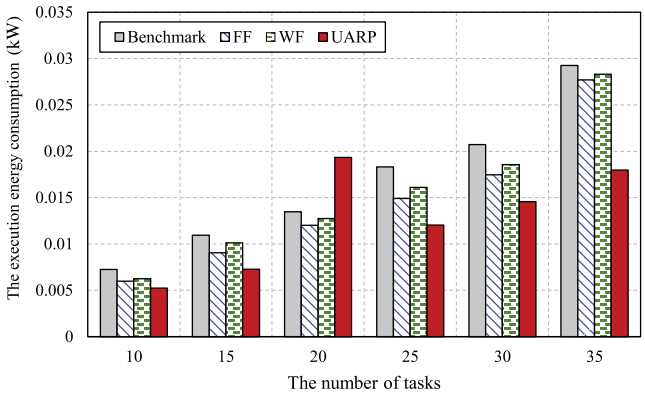


FIGURE 8 Comparison of the execution energy consumption.

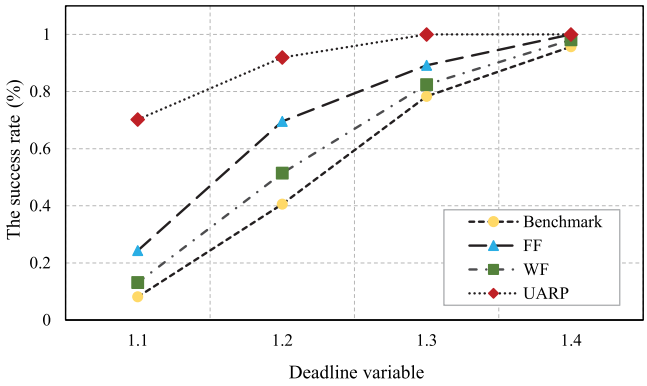


FIGURE 9 Comparison of the success rate.

scheduling strategy is determined. In FF and WF, one edge node needs to continuously send request messages to surrounding nodes to achieve their states, which is a large overhead. Therefore, our method consumes less energy than other two methods. Figure 8 shows the comparison of the execution energy consumption for workflows with different number of tasks. Compared to other methods, it is obvious that UARP consumes less energy to perform the computing tasks in the workflow. Moreover, the execution energy consumption accounts for a large proportion of the total energy consumption.

4.3.3 | Comparison on success rate

The success rate is determined by the actual completion time and the deadline of the tasks. It means that if one task is completed earlier, the success rate will increase. The deadline is defined in Section 3, and the deadline parameter is set to 1.1, 1.2, 1.3, and 1.4. The comparison of the success rate is shown in Figure 9. Since our method has less completion time, the success rate of UARP is higher than the other two methods. If the deadline for the tasks are large, that is, the tasks are not strict with time, the uncertainty has little effect on the tasks. As the deadline increases, the success rates of these methods eventually have similar values.

5 | RELATED WORK

In the 5G networks, with the increasing amount of data to be processed, the workflows that need to be managed grow rapidly. Workflow scheduling became a focus of mobile communication research in recent years.[23] In order to guarantee the QoS and user experience, the uncertainty in workflow scheduling is a problem we need to consider. Currently, most of the existing researches on workflow management system (WFMS) are based on a stable and smooth workflow, and many cases of the abnormal workflow have to be handled manually, which needs to be solved urgently.[24-27]

In Reference 24, Jinzhu Hu et al introduced a software, named Agent, which has the function to monitor the status of workflow in real-time and interrupts its execution when the workflow is abnormal. Through the ability of independent learning and imitation, Agent gives the current correct scheduling strategy and take corresponding actions intelligently, so as to make the workflow go smoothly. Generally, the available bandwidth is difficult to express in a deterministic value because multiple devices share communication infrastructure. Therefore, the deterministic scheduler is not able to provide a correct and effective scheduling strategy. In Reference 25, a scheduling strategy, named RobWE is introduced, and the strategy is based on particle swarm optimization, which provides a robust scheduling strategy under inaccurate estimation of available bandwidth. In Reference 26, WenAn Tan et al developed a workflow scheduling algorithm for face trust service, which uses direct trust and recommendation of trust as trust metrics. In order to address the uncertainty in the workflow, Weidong Zhao et al[27] proposed a frequent pattern mining algorithm based on the Apriori algorithm, whose model quantifies the intensity of the logical relationship between activities and predicted trends.

In edge computing, in addition to addressing the uncertainty in the workflow, other factors, including energy consumption and the completion time, also need to be considered.[28-30] Currently, there are many studies have optimized energy consumption and the completion time.[4,31-33] To improve energy efficiency in a computing environment, the virtual machines (VMs) are also required to host applications.[4] In Reference 31, San-Yih Hwang et al divided the function of the distributed WFMS into several components, and these components are distributed to different sites for execution. Then, the components are cooperated with each other to communicate the underlying business processes and San-Yin Hwang et al discuss how to distribute WFMS components. Hamid Mohammadi Fard et al[32] proposed a new algorithm named Robust MOHEFT (R-MOHEFT) to solve the uncertainty of neutron source performance in cloud infrastructure and the method in this paper approximates the pareto optimal workflow scheduling solution set in terms of computing time and robustness. Huangke Chen et al[33] developed an uncertain-aware online scheduling algorithm to schedule deadlines for dynamic multiple workflows and the computing environment is improved by minimizing the propagation of uncertainty in workflow scheduling.

In traditional edge networks, workflow scheduling lacks the flexibility to dynamically allocate resources on demand.[34] Therefore, SDN and MEC are able to be integrated, with user workflow captured through SDN data platform, and appropriate resources are flexibly combined to ensure the autonomous execution of a data-intensive scientific workflow.[12,35,36] Currently, SDN has been integrated with edge computing in many research to provide some advantages, including the utilization of decentralized resources, the reduction of time and privacy protection.[12,37-39]

In order to overcome the performance limitations of traditional SDN, Almohaimeed A et al proposed a new architecture for SDN-based network edge control,[12] and deployed edge controllers near network devices.[37] Integrating the centralized control programmability of SDN with the robustness of the distributed protocol, the advantages and disadvantages of each method are discussed. In,[40] Lianyong Qi et al proposed a time-aware distributed service recommendation with privacy protection, which is encrypted by local sensitive hash. Chung-Ming Huang et al[38] applied the SDN within the MEC architecture to solve the complex unloading problem of vehicle-to-vehicle in the vehicle ad-hoc network.

However, SDN is often integrated with edge computing to enable efficient utilization of computing and storage resources. As far as we know, the uncertainty of workflow scheduling in edge computing is rarely considered by existing researches. In order to solve the uncertainty, a SDN-based edge computing framework and a dynamic resource provisioning are proposed in our paper.

6 | CONCLUSION AND FUTURE WORK

Nowadays, due to the long distance between data centers and mobile devices, edge computing emerges to provide low latency services for mobile devices. In addition, a set of sequential tasks in the mobile device, organized as the workflow, are migrated to the edge nodes for further processing. The edge nodes are uniformly managed by multiple SDN controllers, and the scheduling strategies are conducted in the proposed SDN-based edge computing framework. However, the process of task execution does not always follow the predetermined strategies, and some unexpected situations occur. Aiming at this challenge, a dynamic resource provisioning method (UARP), implemented in the proposed framework, is proposed to address the uncertainty. UARP optimizes two important goals simultaneously, and balanced scheduling strategies are generated. Experimental results and analysis demonstrate the effectiveness of our proposed method. In the future, more kinds of uncertainties are considered and UARP will be experimented in the real world.

ACKNOWLEDGMENTS

This research is supported by the National Natural Science Foundation of China under grant no. 61702277 and no. 61702442. This work is also supported by the Priority Academic Program Development of Jiangsu Higher Education Institutions (PAPD) fund.

ORCID

Xiaolong Xu https://orcid.org/0000-0003-4879-9803

REFERENCES

1. Wang L, Zhen H, Fang X, Wan S, Ding W, Guo Y. A unified two-parallel-branch deep neural network for joint gland contour and segmentation learning. *Futur Gener Comput Syst*. 2019;100:316-324.
2. Xu Y, Qi L, Dou W, Yu J. Privacy-preserving and scalable service recommendation based on simhash in a distributed cloud environment. *Complexity*. 2017;2017:1-9.
3. Li W, Liu X, Liu J, Chen P, Wan S, Cui X. On improving the accuracy with auto-encoder on conjunctivitis. *Appl Soft Comput*. 2019;81:105489.
4. Qi L, Chen Y, Yuan Y, Fu S, Zhang X, Xu X. A QoS-aware virtual machine scheduling method for energy conservation in cloud-based cyber-physical systems. *World Wide Web*. 2019;1-23.
5. Gong W, Qi L, Xu Y. Privacy-aware multidimensional mobile service quality prediction and recommendation in distributed fog environment. *Wirel Commun Mob Comput*. 2018;2018:1-8.
6. Zhang Y, Cui G, Deng S, Chen F, Wang Y, He Q. Efficient query of quality correlation for service composition. *IEEE Trans Serv Comput*. 2018.
7. Zhang Y, Wang K, He Q, et al. Covering-based web service quality prediction via neighborhood-aware matrix factorization. *IEEE Trans Serv Comput*. 2019.
8. Kreutz D, Ramos F, Verissimo P, Rothenberg CE, Azodolmolky S, Uhlig S. Software-defined networking: a comprehensive survey. 2014. arXiv preprint arXiv:1406.0440.
9. Qi L, He Q, Chen F, et al. Finding all you need: web APIs recommendation in web of things through keywords search. *IEEE Trans Comput Soc Syst*. 2019;6:1063-1072.
10. Wang S, Zhou A, Bao R, Chou W, Yau SS. Towards green service composition approach in the cloud. *IEEE Trans Serv Comput*. 2018.
11. Gao Z, Xuan H-Z, Zhang H, Wan S, Choo K-KR. Adaptive fusion and category-level dictionary learning model for multi-view human action recognition. *IEEE Internet Things J*. 2019.
12. Almohaimeed A, Asaduzzaman A. Introducing edge controlling to software defined networking to reduce processing time. Paper presented at: Proceedings of the 2019 IEEE 9th Annual Computing and Communication Workshop and Conference; 2019:0585-0590; IEEE.
13. Wan S, Zhao Y, Wang T, Gu Z, Abbasi QH, Choo K-KR. Multi-dimensional data indexing and range query processing via voronoi diagram for internet of things. *Futur Gener Comput Syst*. 2019;91:382-391.
14. Liu J, Wang W, Li D, Wan S, Liu H. Role of gifts in decision making: an endowment effect incentive mechanism for offloading in the IoV. *IEEE Internet Things J*. 2019.
15. JoSEP AD, KAtz RAD, KonWinSKi AD, Gunho LEE, PAttERSon DAVD, RABKin AREL. A view of cloud computing. *Commun ACM*. 2010;53(4):50-58.
16. Dejun J, Pierre G, Chi CH. EC2 performance analysis for resource provisioning of service-oriented applications. Paper presented at: Proceedings of the Service-Oriented Computing. ICSOC/ServiceWave 2009 Workshops; 2009:197-207; Springer, Berlin, Heidelberg.
17. Xu X, Liu Q, Luo Y, et al. A computation offloading method over big data for IoT-enabled cloud-edge computing. *Futur Gener Comput Syst*. 2019;95:522-533.
18. Yuan Y, Xu H, Wang B. An improved NSGA-III procedure for evolutionary many-objective optimization. Paper presented at: Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation; 2014:661-668; ACM.
19. Qin Q, Poularakis K, Iosifidis G, Kompella S, Tassiulas L. SDN controller placement with delay-overhead balancing in wireless edge networks. *IEEE Trans Netw Serv Manag*. 2018;15(4):1446-1459.
20. Wang S, Zhou A, Yang F, Chang RN. Towards network-aware service composition in the cloud. *IEEE Trans Cloud Comput*. 2016.
21. Baktir AC, Ozgovde A, Ersoy C. How can edge computing benefit from software-defined networking: a survey, use cases, and future directions. *IEEE Commun Surv Tutor*. 2017;19(4):2359-2391.
22. Meng S, Wang S, Wu T, et al. An uncertainty-aware evolutionary scheduling method for cloud service provisioning. Paper presented at: Proceedings of the 2016 IEEE International Conference on Web Services (ICWS); 2016:506-513; IEEE.
23. Wang PW, Ding ZJ, Jiang CJ, Zhou MC, Zheng YW. Automatic web service composition based on uncertainty execution effects. *IEEE Trans Serv Comput*. 2015;9(4):551-565.
24. Hu J, Zhang C, Zhang Y, Zhao D, Chen J, Fang C. Research of exception handling in workflow management system based on agents group. Paper presented at: Proceedings of the 2008 3rd International Conference on Innovative Computing Information and Control; 2008:232-232; IEEE.
25. Genez TAL, Bittencourt LF, Sakellariou R, Madeira ERM. A robust scheduler for workflow ensembles under uncertainties of available bandwidth. Paper presented at: Proceedings of the 2017 IEEE 10th International Conference on Cloud Computing (CLOUD); 2017:504-511; IEEE.
26. Tan WA, Yong S, Xia LL, Lu GZ, Wang T. A trust service-oriented scheduling model for workflow applications in cloud computing. *IEEE Syst J*. 2013;8(3):868-878.
27. Zhao W, Ye M. A workflow frequent pattern mining algorithm. Paper presented at: Proceedings of the 2007 International Conference on Wireless Communications, Networking and Mobile Computing; 2007:3685-3688; IEEE.
28. Viswanathan H, Lee EK, Pompili D. A multi-objective approach to real-time in-situ processing of mobile-application workflows. *IEEE Trans Parall Distr Syst*. 2016;27(11):3116-3130.
29. Genez TAL, Bittencourt LF, Fonseca NLS, Madeira ERM. Estimation of the available bandwidth in inter-cloud links for task scheduling in hybrid clouds. *IEEE Trans Cloud Comput*. 2015;7(1):62-74.
30. Xie X, Tianwei Y, Xiao Z, Cheng X. Research on trust model in container-based cloud service. *Comput Mater Con*. 2018;56(2):273-283.
31. Hwang SY, Yang CT. Component and data distribution in a distributed workflow. Paper presented at: Proceedings of the 1998 Asia Pacific Software Engineering Conference; 1998:244-251; IEEE.
32. Fard HM, Ristov S, Prodan R. Handling the uncertainty in resource performance for executing workflow applications in clouds. Paper presented at: Proceedings of the 2016 IEEE/ACM 9th International Conference on Utility and Cloud Computing; 2016:89-98; IEEE.
33. Chen H, Zhu X, Liu G, Pedrycz W. Uncertainty-aware online scheduling for real-time workflows in cloud service environment. *IEEE Trans Serv Comput*. 2018.

34. Zhang J, Xie N, Zhang X, Yue K, Li W, Kumar D. Machine learning based resource allocation of cloud computing in auction. *Comput Mater Con*. 2018;56(1):123-135.
35. Gharbaoui M, Contoli C, Davoli G, et al. Demonstration of latency-aware and self-adaptive service chaining in 5G/SDN/NFV infrastructures. Paper presented at: Proceedings of the 2018 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN); 2018:1-2; IEEE.
36. Yang Z, Huang Y, Li X, et al. Efficient secure data provenance scheme in multimedia outsourcing and sharing. *Comput Mater Con*. 2018;56(1):1-17.
37. Poularakis K, Qin Q, Nahum E, Rio M, Tassiulas L. Bringing SDN to the mobile edge. Paper presented at: Proceedings of the 2017 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computed, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI); 2017:1-6; IEEE.
38. Huang C-M, Chiang M-S, Dao D-T, Su W-L, Xu S, Zhou H. V2V data offloading for cellular network based on the software defined network (SDN) inside mobile edge computing (MEC) architecture. *IEEE Access*. 2018;6:17741-17755.
39. Aggarwal C, Srivastava K. Securing IoT devices using SDN and edge computing. Paper presented at: Proceedings of the 2016 2nd International Conference on Next Generation Computing Technologies; 2016:877-882; IEEE.
40. Qi L, Wang R, Hu C, Li S, He Q, Xu X. Time-aware distributed service recommendation with privacy-preservation. *Inf Sci*. 2019;480:354-364.

How to cite this article: Xu X, Cao H, Geng Q, Liu X, Dai F, Wang C. Dynamic resource provisioning for workflow scheduling under uncertainty in edge computing environment. *Concurrency Computat: Pract Exper*. 2020;e5674. https://doi.org/10.1002/cpe.5674